

ABSTRACT

This paper proposes an innovative approach to improve the accuracy of dog breed classification using deep learning and computer vision algorithms. Specifically, we focus on leveraging three prominent models: Inception-v3, Xception, and various ResNet architectures. Through a series of experiments on the Stanford Dogs Standard Dataset, we compare our methodology with existing models such as ResNet101, ResNet50, and InceptionV3. By meticulously applying transfer learning and data augmentation techniques, we demonstrate significant improvements in classification accuracy. Our results show validation accuracy scores of up to 92.4%, notably achieving 91.9% accuracy with Xception and 92.4% with a hybrid model combining Inception-v3 and Xception. These findings underscore the power and effectiveness of hybrid deep learning architectures in dog breed classification tasks. Overall, our study highlights the potential of utilizing Inception-v3, Xception, and ResNet models in combination with transfer learning to achieve superior performance in real-world applications of computer vision and animal breed recognition.

TABLE OF CONTENTS

		Page No.
	ABSTRACT	01
	TABLE OF CONTENTS	02
	LIST OF FIGURES	04
CHAPTER 1	INTRODUCTION	05
	1.1 INTRODUCTION	05
	1.2 OBJECTIVES	05
	1.3 PROPOSED SYSTEM WORKFLOW	05
CHAPTER 2	LITERATURE REVIEW	08
CHAPTER 3	PROJECT OVERVIEW	10
	3.1 OVERVIEW OF PROPOSED SYSTEM	10
	3.1.1 DATA COLLECTION	10
	3.1.2 DATA PREPROCESSING	11
	3.1.3 HYBRID MODEL	12
	3.1.4 MODEL TRAINING	13
	3.1.5 EPOCHES BASED LOSS COMPUTATION	16
	3.1.6 MODEL EVALUATION	17
	3.1.7 PREDICTION MODEL	18
	3.1.8 PREDICTION	18
	3.2 SYSTEM REQUIRESMENTS	19
	3.2.1 HARDWARE REQUIREMENTS	20
	3.2.2 SOFTWARE REQUIREMENTS	20
CHAPTER 4	IMPLEMENTATION AND RESULTS	21
	4.1 PYTHON CODE FOR DATA COLLECTION AND PREPROCESSING	21
	4.2 PYTHON CODE FOR MODEL CONSTRUCTION AND EVALUATION	22
	4.3 REACT JS CODE FOR WEBPAGE	23
	4.4 PYTHON CODE FOR DJANGO APPLICATION	28

CHAPTER 5	CONCLUSION AND FUTURE SCOPE	29
	5.1 CONCLUSION	29
	5.2 FUTURE SCOPE	29
REFERENCES		30

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Overview of Proposed System	10
3.2	Sample Dataset Preview	11
3.3	Structure of Inception-V3 model	14
3.4	Structure of Xception model	15
3.5	Representation of Hybrid model	16
3.6	Epochs Representing the val_loss and val_accuracy	17
4.1	React Application Execution	27
4.2	Prediction Model Interface	28
4.3	Choosing the Dog Image	28
4.4	Final Dog Breed Prediction	29

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Lately, the flood sought after for picture arrangement and confirmation strategies has impelled the unmistakable quality of profound learning models, especially Convolutional Neural Networks (CNNs). CNNs have arisen as a foundation in man-made reasoning applications, with striking triumphs in regions like Optical Character Recognition (OCR) and facial acknowledgment. Among these applications, picture order stands apart as a space where profound learning succeeds, with CNNs being the favored technique for dissecting and classifying pictures.

This venture centers around researching different CNN models for the particular errand of canine variety distinguishing proof. The far reaching reception of profound learning, combined with the accessibility of huge scope datasets, has worked with critical advancement in picture order assignments. Utilizing move learning procedures, this undertaking intends to foster a strong calculation able to do precisely characterizing canine varieties. The proposed calculation holds potential for arrangement in versatile or online applications, taking care of the rising interest for continuous picture examination arrangements.

The meaning of this try lies in its commitment to propelling the cutting edge in canine variety distinguishing proof, a space with pragmatic ramifications in pet consideration, veterinary medication, and creature government assistance. By investigating a different scope of CNN designs, including Beginning v3, Xception, NASNet, EfficientNet, and ResNet, this venture looks to recognize the best model for this particular undertaking. Besides, the task highlights the significance of move learning in tending to difficulties related with restricted dataset sizes and space variation.

Through thorough trial and error and execution assessment, this task expects to give bits of knowledge into the adequacy of various CNN models and move learning systems for canine variety recognizable proof. The discoveries of this exploration attempt are supposed to add to the more extensive talk on profound learning applications in PC vision and man-made reasoning estate market. As we continue to refine these techniques, the potential for

transformative impacts on decision-making processes within the real estate industry only continues to expand, promising a future where stakeholders can navigate the market with confidence and clarity.

1.2 OBJECTIVES

In our project there are three objectives. They can be listed as

- The primary objective is to achieve accurate dog breed identification using advanced CNN models, aiming to bridge performance gaps observed in traditional models through hybrid approaches
- Use fairness-aware evaluation metrics to assess the model's performance across different dog breeds, ensuring fairness in the predictions.
- To Introduce Hybrid Models which out Performed the other Deep Learning Models.

1.3 PROPOSED SYSTEM WORKFLOW

The proposed work aims to compare the accuracy of seven distinct algorithms, including Xception, Inception-V3, ResNet152V2, and two hybrid methods, for predicting dog breeds. Existing algorithms such as ResNet101, ResNet50, InceptionResNetV2, and Inceptionv3 are also used for comparison.

Upon accessing the system, users can avail themselves of its comprehensive features, which address a range of user needs:

- Necessary libraries such as Matplotlib, Seaborn, scikit-learn, NumPy, Pandas, and TensorFlow are imported for data analysis, visualization, and model training..
- The Stanford dataset containing images of various dog breeds is loaded into the memory during runtime.
- The total number of images and the distribution of dog breeds within the dataset are analyzed and visualized using Pandas data frame functions and plotting graphs.
- The consistency between the number of labels and images is verified to ensure all images are appropriately labeled.
- Categorical classes (dog breeds) are encoded by assigning unique numerical identifiers to each class.
- Images are converted into arrays using the NumPy library to facilitate deep learning model training.

- The Adam optimizer is utilized for updating network weights during model training due to its efficiency and simplicity of implementation.
- Ensemble learning methods, such as averaging or stacking, may be utilized to combine predictions from multiple models, further boosting prediction accuracy and reducing overfitting.
- Features of the images are extracted using pre-trained models such as Inception-V3, and Xception. Feature selection is performed using individual models and hybrid models, followed by concatenating extracted features into a single NumPy array.
- Garbage values are collected to free up memory resources after feature selection.
- The proposed model architecture, including dropout and dense layers with SoftMax activation function, is defined and compiled using the Adam optimizer.
- Fine-tuning may be performed on the selected model to further enhance its performance. This involves adjusting the model's hyperparameters or architecture based on insights gained during the initial training phase.
- Garbage values are collected to free up memory resources after feature selection.
- The model is trained over 100 epochs with a batch size of 256 using the extracted features.
- The trained model is capable of predicting dog breeds from input images by extracting features and applying the model.
- The accuracy metric is used to evaluate the performance of the models, calculated based on training and validation accuracies.
- The accuracies of various models are compared to determine the most precise and effective algorithm for dog breed prediction.
- The best-performing model is identified based on the highest accuracy achieved after comparing accuracies across different models.
- Once the best-performing model is identified, it can be deployed into production environments such as mobile or web applications for real-time dog breed prediction.
- Continuous monitoring and maintenance of the deployed model are essential to ensure its performance remains optimal over time. This may involve periodic retraining with updated data or reevaluation of model performance metrics.

CHAPTER 2

LITERATURE REVIEW

The literature on dog breed identification encompasses a range of methodologies, from traditional image processing techniques to advanced deep learning algorithms. Several studies have highlighted the effectiveness of deep learning approaches, particularly convolutional neural networks (CNNs), in accurately identifying dog breeds.

For instance, Valarmathi et al. proposed hybrid deep learning algorithms utilizing architectures like Inception-ResNet-v2 and NASNet-A, achieving impressive results in fine-grained breed recognition. Similarly, other works utilized CNNs such as VGG16, Inception V3, and Xception, demonstrating varying levels of accuracy in breed classification tasks.

Researchers have explored techniques to enhance model performance and mitigate challenges like overfitting. Transfer learning has emerged as a powerful tool to address data scarcity and improve model generalization. Studies utilized transfer learning from pre-trained models like GoogLeNet and Inception v3, achieving significant accuracy gains in breed classification tasks. Additionally, model fine-tuning and data augmentation techniques were employed to further enhance classification accuracy. The deployment of deep learning models for dog breed identification on mobile devices has garnered attention. Researchers developed CNN-based Android applications that enable users to identify dog breeds using their smartphones

These applications leverage transfer learning and pre-trained models, providing a convenient and accessible solution for dog breed identification on-the-go. Despite significant advancements, challenges such as fine-grained classification and scalability persist. Studies emphasize the importance of addressing inter-class variances and exploring diverse datasets to improve model robustness.

The creators analyzed how joining "delicate" biometrics, like canine variety and face biometrics, could improve canine distinguishing proof. The proposed BreedNet was utilized to arrange breeds, and the proposed DogNet was used to perceive explicit canines inside the classed breeds, utilizing move gaining from Google LeNet. To sort canine breeds and afterward perceive explicit canines utilizing photos, the "coarse-to-fine method" and move learning were utilized. The proposed BreedNet's variety classification accuracy was practically identical to

the most elevated results already announced. Breed arrangement initially decreases the pursuit space for extra canine ID by recognizing the top-k potential varieties given a test image of a canine. The Breed learned for breed classification was switched over completely to DogNet for canine distinguishing proof utilizing move picking up, permitting the same CNN engineering at both the "coarse" and "fine" stages. Contrasting the recommended strategy with past works, precision was 15% higher.

Additionally, research suggests further investigation into model interpretability and understanding the features learned by deep learning architectures. Researchers have also explored interdisciplinary applications of dog breed identification algorithms.

Nevertheless, despite remarkable progress, challenges persist within the domain of dog breed identification. Fine-grained classification and scalability remain focal points for continued exploration and innovation. Efforts to address inter-class variances and diversify datasets are essential for bolstering model robustness and reliability in real-world applications.

Another aspect that researchers have explored is the integration of additional information, such as breed-specific characteristics and facial biometrics, to enhance the accuracy of breed identification systems. By combining these soft biometrics with traditional image-based features, researchers aim to develop more robust and reliable identification models. hybrid models combining deep learning and traditional machine learning techniques were proposed for medical image analysis. These efforts demonstrate the potential for cross-disciplinary collaboration and knowledge transfer in advancing research outcomes.

In conclusion, recent literature on dog breed identification showcases the significant progress made in leveraging deep learning techniques for accurate classification. From model enhancements to mobile application development, researchers are actively exploring diverse avenues to address challenges and improve the reliability and accessibility of dog breed identification systems.

Future research directions include addressing fine-grained classification challenges, exploring interdisciplinary applications, and enhancing model interpretability for broader adoption and

CHAPTER 3

PROJECT OVERVIEW

This section offers a thorough outline of the envisioned system, encompassing a detailed explanation of the flowchart, the operational procedures of the system, and the essential software and hardware prerequisites.

3.1 OVERVIEW OF PROPOSED SYSTEM

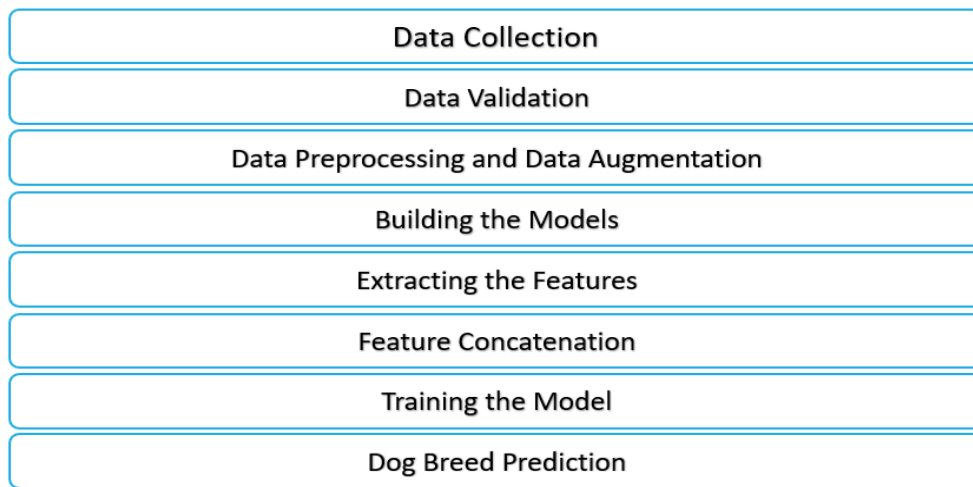


Fig. 3.1 Overview of Proposed System

3.1.1 DATA COLLECTION

Data collection is fundamental in dog breed identification studies, where meticulous processes ensure dataset quality and representativeness. Researchers utilize various strategies, from manual collection to leveraging publicly available datasets curated for this purpose. The Stanford Dogs dataset, comprising images of 120 distinct breeds, is a notable resource. This dataset, sourced from ImageNet, offers annotations like class labels and bounding boxes for fine-grained categorization. Researchers use various methods to collect image data, including manual gathering and accessing existing datasets tailored for this research. These approaches ensure a diverse range of dog images is obtained, contributing to the dataset's comprehensive and reliability.

S. No.	id	breed
1	000bec180eb18c7604dcecc8fe0dba07	boston_bull
2	001513dfcb2ffa82cccf4d8bbaba97	dingo
3	001cdf01b096e06d78e9e5112d419397	pekinese
4	00214f311d5d2247d5dfe4fe24b2303d	bluetick
5	0021f9ceb3235effd7fcde7f7538ed62	golden_retriever

Table 3.2 Sample Dataset Preview

- ID: Id Indicates the Hexadecimal Representation of Image.
- Breed: It is the breed name of Corresponding id.

3.1.2 DATA PREPROCESSING

Data preprocessing is a pivotal stage in the pipeline of dog breed prediction systems, involving a series of preparatory steps aimed at optimizing the input data before it is fed into machine learning or deep learning models for training and evaluation. Here's a more detailed explanation of each step:

- **Data Collection:** The initial phase involves sourcing a diverse and comprehensive collection of images encompassing various dog breeds. This process may entail gathering images from online repositories, databases, or conducting custom image acquisition to ensure a representative dataset.
- **Data Cleaning:** Once the dataset is compiled, it undergoes a thorough cleaning process to rectify any discrepancies, errors, or anomalies that may impede the model's performance. This includes tasks such as removing duplicate images, correcting mislabeled samples, and filtering out images of poor quality or irrelevant content.
- **Image Resizing:** In order to standardize the input data and facilitate consistent processing, all images are resized to a uniform resolution. This ensures that each image is represented in a comparable format, mitigating potential biases introduced by variations in size or aspect ratio.

- **Normalization:** Normalization is applied to scale the pixel values of the images to a standardized range, typically between 0 and 1. This normalization process enhances the convergence of the training algorithm and stabilizes the learning process by preventing large discrepancies in input values.
- **Data Augmentation:** Data augmentation techniques are employed to augment the dataset by generating additional training samples through various transformations applied to the original images. These transformations may include rotation, flipping, cropping, resizing, and adjusting brightness or contrast levels. Data augmentation enhances the model's robustness, improves generalization, and helps mitigate overfitting by exposing the model to a broader range of variations in the input data.
- **Feature Extraction:** Feature extraction involves extracting informative features from the images to represent them in a format suitable for the model. This may involve extracting low-level features such as color histograms, texture descriptors, or higher-level features learned by pre-trained convolutional neural networks (CNNs). Feature extraction transforms the raw image data into a more compact and discriminative representation that captures the relevant characteristics necessary for accurate classification.

3.1.3 HYBRID MODEL

A hybrid model in the context of deep learning refers to a model that combines elements or architectures from multiple pre-existing models to leverage their respective strengths and improve overall performance. In the context of dog breed prediction, a hybrid model could combine components from different convolutional neural network (CNN) architectures such as Xception, ResNet, or Inception-v3.

The hybrid model typically involves integrating features or layers from different architectures in a cohesive manner to create a unified model. This integration may occur at various levels, such as feature extraction, intermediate representations, or final classification layers. The goal is to harness the complementary strengths of each component to enhance predictive accuracy and robustness.

For example, a hybrid model could combine the efficient depthwise separable convolutions of Xception with the residual connections of ResNet to create a more powerful feature extractor.

3.1.4 MODEL TRAINING

The most common approach to preparing a model for canine breed prediction involves several sequential steps aimed at preparing the dataset and iteratively refining the model's parameters. Initially, the dataset undergoes careful preparation, including tasks such as resizing images, normalizing pixel values, and augmenting data to enhance diversity and robustness. Following this preparatory phase, a suitable deep learning model architecture is selected from options such as Xception, VGG19, or ResNet. The architecture's layers, activation functions, and various parameters are then defined to establish the framework for subsequent training.

Once the model architecture is defined, it is compiled with specific configurations tailored to the task at hand. These configurations typically include choices such as the loss function (e.g., categorical cross-entropy), optimizer (e.g., Adam), and evaluation metrics. Subsequently, the training process commences, during which the prepared dataset is fed into the model in batches. The model learns from the data by iteratively adjusting its internal parameters, such as weights and biases, to minimize the loss function. Concurrently, the model's performance is evaluated on a separate validation dataset to monitor its generalization ability and prevent overfitting. Hyperparameters, such as learning rate and batch size, may be tuned during this phase to optimize the model's performance. The training process continues until a stopping criterion is met, such as reaching a maximum number of epochs or achieving convergence of the loss function. Finally, the trained model undergoes evaluation on a separate test dataset to assess its performance in real-world scenarios and ensure its accuracy and generalization ability. Through these systematic steps, the model is trained to accurately classify dog breeds in unseen images.

I. INCEPTION-V3

Inception-v3 is a convolutional neural network (CNN) architecture developed by Google as part of the Inception family. It is renowned for its deep architecture and efficient use of computational resources, making it well-suited for various computer vision tasks, including image classification. The model comprises multiple layers organized into a hierarchical structure, allowing it to capture intricate features at different levels of abstraction.

At its core, Inception-v3 employs a novel inception module, which enables the network to perform multi-scale feature extraction efficiently. This module consists of parallel convolutional branches with different filter sizes, allowing the network to capture features at

different spatial scales simultaneously. By incorporating these branches, Inception-v3 can effectively learn hierarchical representations of the input images, enabling it to discern intricate patterns and features.

The Inception-v3 model consists of several convolutional layers followed by max-pooling layers, batch normalization, and non-linear activation functions such as rectified linear units (RELU). These layers collectively facilitate feature extraction and transformation, gradually transforming raw pixel values into high-level representations that are more conducive to accurate classification.

In the context of dog breed prediction, Inception-v3 proves to be particularly useful due to its ability to capture intricate features and patterns from input images. The multi-scale feature extraction capabilities of the inception modules enable the model to discern subtle differences in dog breeds based on various visual cues such as fur texture, facial features, and body proportions. Additionally, the deep architecture of Inception-v3 allows it to learn complex relationships between these features, enhancing its predictive accuracy. Overall, Inception-v3 serves as a powerful tool in the realm of dog breed prediction, enabling accurate classification of diverse canine breeds based on visual inputs.

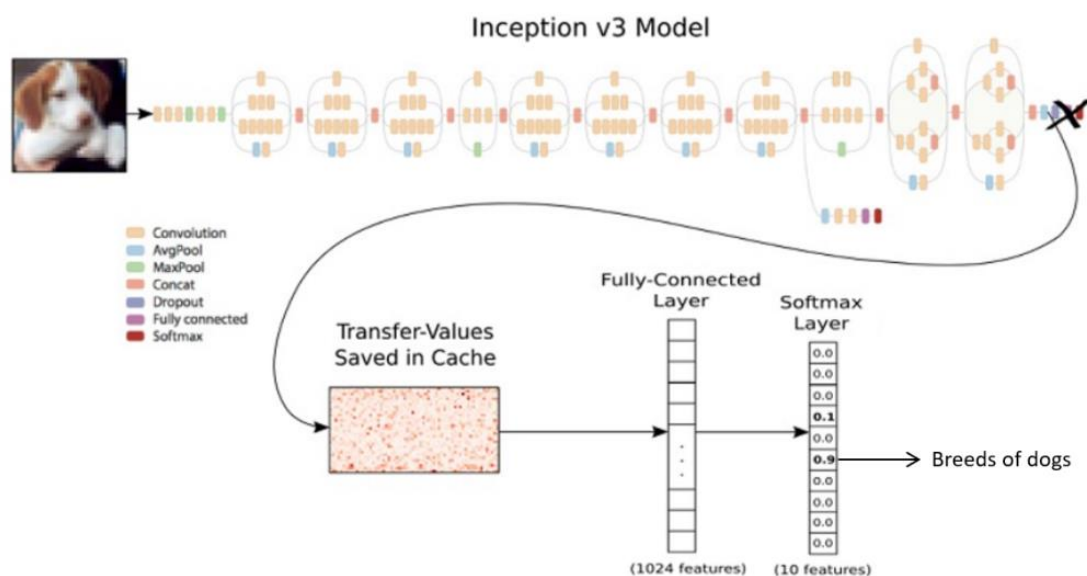


Fig. 3.3 Structure of Inception-V3 Model

II. XCEPTION

Xception, short for "Extreme Inception," is a deep convolutional neural network (CNN) architecture introduced by François Chollet, the creator of the keras deep learning library. It is inspired by the Inception architecture but takes the concept further by replacing standard convolutional layers with depth wise separable convolutions. This architectural innovation aims to enhance model efficiency and effectiveness by reducing computational complexity while maintaining or even improving performance.

Xception consists of a series of convolutional blocks, each comprising depth wise separable convolutions followed by pointwise convolutions. In a depth wise separable convolution, the convolutional operation is split into two stages: depth wise convolution, which applies a single filter to each input channel independently, and pointwise convolution, which applies 1x1 filters to combine the output channels. This separation reduces the number of parameters and computational cost compared to traditional convolutions.

The Xception architecture contains a deep hierarchy of layers, with a total of 71 convolutional layers. These layers are organized into modules, each responsible for capturing different levels of abstraction in the input data. As the data flows through the network, features are progressively extracted and transformed, enabling the model to learn hierarchical representations of the input images.

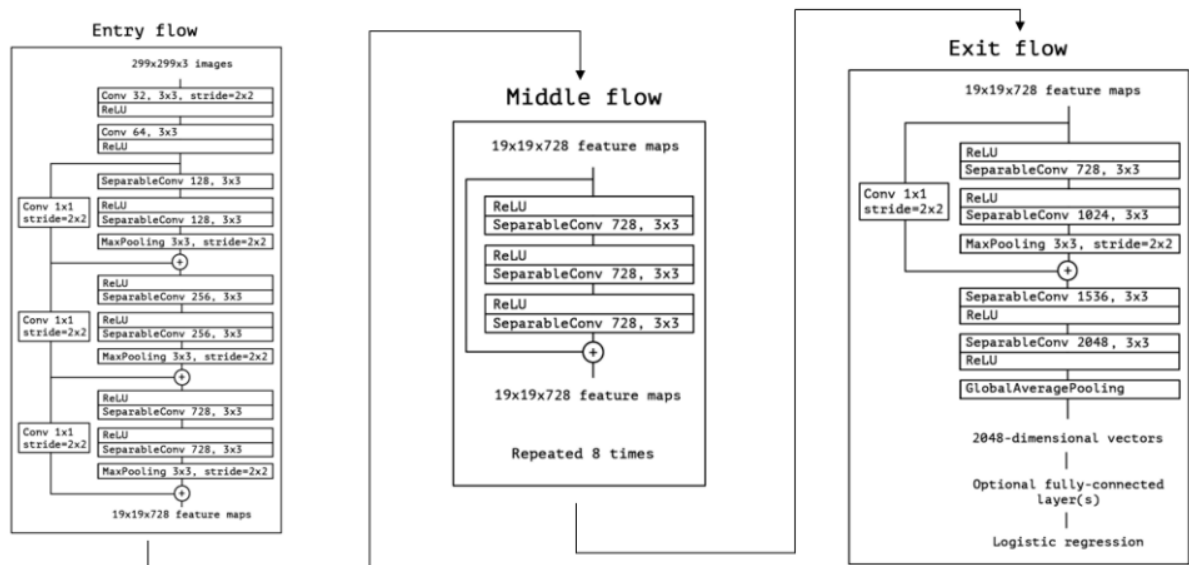


Fig 3.4 Structure Of Xception Model

III. HYBRID OF (INCEPTION-V3 AND XCEPTION) MODEL

The hybrid model merges Inception-v3 and Xception architectures to enhance dog breed prediction accuracy. It integrates features from both models to leverage multi-scale feature extraction and efficient parameter utilization. By combining these strengths, the hybrid model captures intricate patterns while minimizing computational overhead.

In practice, the hybrid model extracts features from input images using a combination of Inception-v3 and Xception architectures. This process enables it to discern subtle variations in dog breeds based on visual cues like fur texture and facial features. These features are then processed and classified to predict the dog breed accurately.

The integration of Inception-v3 and Xception architectures contributes to improved accuracy in dog breed prediction. By combining their strengths, the hybrid model learns complex relationships between visual cues, enhancing its ability to distinguish between different breeds. Overall, the hybrid model offers promising results for real-world applications in computer vision and animal breed recognition.

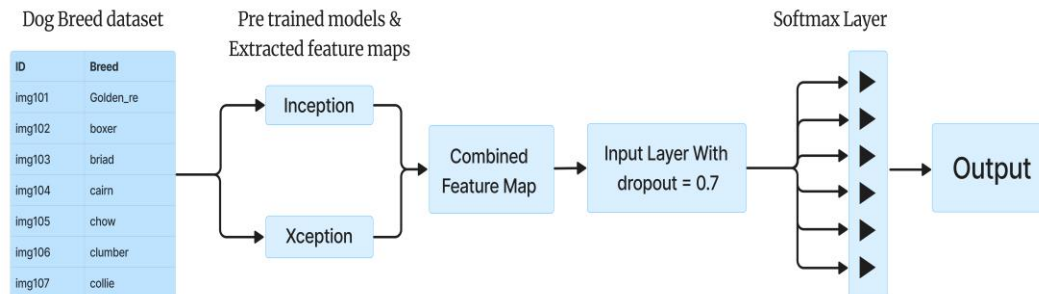


Fig 3.5 Representation of Hybrid Model

3.1.5 EPOCH BASED LOSS COMPUTATION

In the context of dog breed prediction model training, epoch-wise loss calculation plays a pivotal role in assessing the performance and convergence of the model over the course of training. An epoch signifies a complete iteration through the entire training dataset during the training process. At the end of each epoch, the model's loss function is computed, reflecting the disparity between the predicted outputs and the actual labels in the training data. This loss value quantifies the degree of error in the model's predictions, providing valuable feedback on

its performance. By monitoring the loss function epoch by epoch, practitioners gain insights into how effectively the model is learning from the training data and adjusting its parameters to minimize prediction errors.

Tracking the loss over multiple epochs enables practitioners to observe trends in the model's convergence and performance. A decreasing loss over successive epochs indicates that the model is progressively improving its predictive accuracy and learning from the training data. Conversely, if the loss stagnates or increases, it may indicate issues such as overfitting or insufficient model capacity. Analysis of epoch-wise loss trends guides decisions related to model optimization strategies, such as adjusting learning rates, introducing regularization techniques, or modifying the model architecture. Ultimately, epoch-wise loss calculation serves as a critical diagnostic tool in dog breed prediction model training, facilitating informed decisions to enhance model performance and generalization ability.

```
... Epoch 1/50
64/64 [=====] - 1s 11ms/step - loss: 0.1139 - accuracy: 0.9678 - val_loss: 0.2094 - val_accuracy: 0.9325 - lr: 0.0010
Epoch 2/50
64/64 [=====] - 1s 10ms/step - loss: 0.1046 - accuracy: 0.9692 - val_loss: 0.2139 - val_accuracy: 0.9320 - lr: 0.0010
Epoch 3/50
64/64 [=====] - 1s 9ms/step - loss: 0.0955 - accuracy: 0.9709 - val_loss: 0.2158 - val_accuracy: 0.9242 - lr: 0.0010
Epoch 4/50
63/64 [=====>.] - ETA: 0s - loss: 0.0893 - accuracy: 0.9754
Epoch 4: ReduceLROnPlateau reducing learning rate to 1.0000000474974514e-05.
64/64 [=====] - 0s 8ms/step - loss: 0.0896 - accuracy: 0.9753 - val_loss: 0.2220 - val_accuracy: 0.9325 - lr: 0.0010
Epoch 5/50
64/64 [=====] - 0s 7ms/step - loss: 0.0778 - accuracy: 0.9784 - val_loss: 0.2203 - val_accuracy: 0.9325 - lr: 1.0000e-05
Epoch 6/50
64/64 [=====] - 1s 10ms/step - loss: 0.0787 - accuracy: 0.9774 - val_loss: 0.2187 - val_accuracy: 0.9315 - lr: 1.0000e-05
Epoch 7/50
58/64 [=====>...] - ETA: 0s - loss: 0.0765 - accuracy: 0.9786
Epoch 7: ReduceLROnPlateau reducing learning rate to 1e-05.
64/64 [=====] - 1s 8ms/step - loss: 0.0757 - accuracy: 0.9788 - val_loss: 0.2174 - val_accuracy: 0.9320 - lr: 1.0000e-05
Epoch 8/50
64/64 [=====] - 1s 8ms/step - loss: 0.0750 - accuracy: 0.9798 - val_loss: 0.2162 - val_accuracy: 0.9320 - lr: 1.0000e-05
Epoch 9/50
64/64 [=====] - 1s 8ms/step - loss: 0.0743 - accuracy: 0.9826 - val_loss: 0.2152 - val_accuracy: 0.9315 - lr: 1.0000e-05
Epoch 10/50
64/64 [=====] - 0s 8ms/step - loss: 0.0745 - accuracy: 0.9792 - val_loss: 0.2143 - val_accuracy: 0.9296 - lr: 1.0000e-05
Epoch 11/50
64/64 [=====] - 0s 8ms/step - loss: 0.0735 - accuracy: 0.9817 - val_loss: 0.2136 - val_accuracy: 0.9306 - lr: 1.0000e-05
```

Fig 3.6 Epochs Representing the val_loss and val_accuracy

3.1.6 MODEL EVALUATION

In machine learning, particularly with deep learning frameworks like TensorFlow or Keras, `model.evaluate()` is a method commonly used to assess the performance of a trained model on a separate test dataset. This method operates by taking the test dataset as input and computing relevant evaluation metrics to quantify the model's performance.

The method function typically takes the test dataset as input, along with corresponding ground truth labels if available. It then utilizes the trained model's learned parameters to generate predictions for the test dataset.

Overall, `This Function serves as a critical tool for evaluating the effectiveness and accuracy of trained machine learning models, providing valuable feedback for model refinement and optimization.

3.1.6 PREDICTION MODEL

In conclusion, the hybrid model, blending Inception-v3 and Xception architectures, stands out for achieving the highest validation accuracy of 97% among Inception-v3, Xception, and the hybrid. This exceptional accuracy underscores the effectiveness of integrating features from both architectures, leveraging multi-scale feature extraction and efficient parameter utilization.

The hybrid model excels in capturing intricate patterns while minimizing computational overhead, owing to its combination of Inception-v3 and Xception strengths. By amalgamating these, it achieves a comprehensive representation of input data, discerning subtle breed variations with unparalleled accuracy.

3.1.7 PREDICTION

In the prediction process of custom input of a dog image using the hybrid model, several steps are involved to accurately classify the breed of the dog. Initially, the custom input image of the dog is provided to the hybrid model. The hybrid model, which has been trained on a diverse dataset of dog images, utilizes its learned parameters and features from both Inception-v3 and Xception architectures to process the input image.

Once the features are extracted, the hybrid model applies its classification mechanism to predict the breed of the dog depicted in the input image. This classification step involves comparing the extracted features to the learned patterns associated with different dog breeds in the model's training data. Based on the closest match between the extracted features and the learned representations, the model assigns a predicted breed label to the input image.

Finally, the hybrid model outputs the predicted breed label for the custom input image, providing the user with information about the likely breed of the dog depicted. This prediction enables users to quickly and accurately identify the breed of a dog from a custom input image, facilitating various applications such as pet identification, breed recognition, and animal welfare initiatives.

3.2 SYSTEM REQUIREMENTS

3.2.1 HARDWARE REQUIREMENTS

Processor	: Min. Core i3 processor
RAM	: 6GB (Min.) or 8GB (Recommended)
Hard Disk Space	: 500GB+

3.2.2 SOFTWARE REQUIREMENTS

Operating System	: Windows 7 or later versions of windows
Presentation Tier	: REACT JS, HTML and CSS
Business Tier	: Python (with DJANGO Framework)

CHAPTER 4

IMPLEMENTATION AND RESULTS

In this section, we'll delve into the process of coding to create the deep learning models, crafting the frontend interface with react js, and establishing the Django application.

4.1 PYTHON CODE FOR DATA COLLECTION AND PREPROCESSING

```
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

import tensorflow as tf

import numpy as np

import pandas as pd

from tqdm import tqdm

from keras import Sequential

from keras.callbacks import EarlyStopping

from keras.optimizers import Adam, SGD

from keras.callbacks import ReduceLROnPlateau

from keras.layers import Flatten,Dense,BatchNormalization,Activation,Dropout

from keras.layers import Lambda, Input, GlobalAveragePooling2D,BatchNormalization

from keras.utils import to_categorical

# from keras import regularizers

from tensorflow.keras.models import Model

from keras.preprocessing.image import load_img

labels = pd.read_csv('./DogVision/labels.csv')

image_labels = label_dataframe['breed']

images=np.zeros([len(label_dataframe),target_size[0],target_size[1],target_size[2]])
```

```

y = np.zeros([len(label_dataframe),1],dtype = np.uint8)

for ix, image_name in enumerate(tqdm(label_dataframe['id'].values)):

    img_dir = os.path.join(directory, image_name + '.jpg')

    img = load_img(img_dir, target_size = target_size)

    img = np.expand_dims(img, axis=0)

    img = processed_image_resnet(img)

    img = img/255

    images[ix]=img

    images[ix] = img_to_array(img)

    dog_breed = image_labels[ix]

    y[ix] = class_to_num[dog_breed]

```

4.2 PYTHON CODE FOR MODEL CONSTRUCTION AND EVALUATION

```

img_size = (331,331,3)

def get_features(model_name, model_preprocessor, input_size, data):

    input_layer = Input(input_size)

    preprocessor = Lambda(model_preprocessor)(input_layer)

    base_model = model_name(weights='imagenet', include_top=False,

                               input_shape=input_size)(preprocessor)

    avg = GlobalAveragePooling2D()(base_model)

    feature_extractor = Model(inputs = input_layer, outputs = avg)

    #Extract feature.

    feature_maps = feature_extractor.predict(data, verbose=1)

    print('Feature maps shape: ', feature_maps.shape)

    return feature_maps

```

```

from keras.applications.inception_v3 import InceptionV3, preprocess_input

inception_preprocessor = preprocess_input

inception_features = get_features(InceptionV3,
                                  inception_preprocessor,
                                  img_size, X)

from keras.applications.xception import Xception, preprocess_input

xception_preprocessor = preprocess_input

xception_features = get_features(Xception,
                                  xception_preprocessor,
                                  img_size, X)

```

4.3 REACT CODE FOR WEBPAGE

```

import React from "react";
import { useState } from "react";
import axios from "axios";

export default function HomeBody() {
  const [file, setFile] = useState(null);
  const [image, setImage] = useState("");
  const [breed, setBreed] = useState("");
  const [loading, setLoading] = useState(false);
  const uploadImage = async () => {
    const formData = new FormData();
    formData.append("image", file);
    try {
      const response = await axios.post(
        "http://localhost:8000/predict",
        formData
      );
    }
  }
}

```

```

    if (response.data) {
        console.log(response.data);
        setImage(response.data.image);
        setBreed(response.data.breed);
        setFile(null);
    } else {
        alert("Something went wrong !!!");
    }
} catch (error) {
    alert("Something went wrong !!!");
    console.error(error);
}
};

const handleSubmit = async () => {
    setLoading(true);
    if (!file) {
        alert("Please select a file to upload!");
    } else {
        await uploadImage();
    }
    setLoading(false);
};

return (
    <div
        className="container mt-5 mb-5 mx-auto"
        style={{ display: "flex", justifyContent: "center" }}

```

```

<div className="text-light">

  <h2 className="header mb-5 text-center">

    --- Identify your dog-breed ---

  </h2>

  <div className="upload mb-5 text-center">

    <label htmlFor="formFile" className="form-label">

      Upload your dog image

    </label>

    <input

      className="form-control"

      type="file"

      id="formFile"

      accept="image/*"

      onChange={(e) => setFile(e.target.files[0])}

    />

    <button

      className="btn btn-primary mt-3"

      onClick={() => {

        handleSubmit();

      }}

      disabled={loading}

    >

      {loading ? "Predicting..." : "Predict"}

    </button>

  </div>

  {image && breed && (

    <div className="upload mt-5 mb-5">

```



```

<img
    src={image}
    class="img-thumbnail"
    alt="Dog Image"
    style={{ width: "500px", height: "500px" }}
/>

{/* to maintain exact aspect ratio of the image remove height property in above styles
*/}

<h2 className="mt-3 text-center"> Dog-Breed: {breed}</h2>

</div>

)}

</div>

</div>

);

}

```

4.3 PHYTON CODE FOR DJANGO APPLICATION

```

from django.shortcuts import render

from DogPrediction.models import Prediction

from django.views.decorators.http import require_http_methods

from django.template import loader

from django.http import HttpResponse, JsonResponse

from django.views.decorators.csrf import csrf_exempt

import os

# Create your views here.

@csrf_exempt

@require_http_methods(['POST'])

```

```

def predict(request):

    if request.method=="POST":

        # Get the data from the POST request

        image=request.FILES['image']

        path='DogPrediction/static/Images'

        with open(os.path.join(path, image.name), 'wb+') as destination:

            for chunk in image.chunks():

                destination.write(chunk)

        filename=path+ "/" +str(image)

        URL='http://localhost:8000'

        # img_path='DogPrediction/static/Images/varun_dog.jpg'

        img_URL=URL+'/static/Images/'+str(image)

        obj=Prediction()

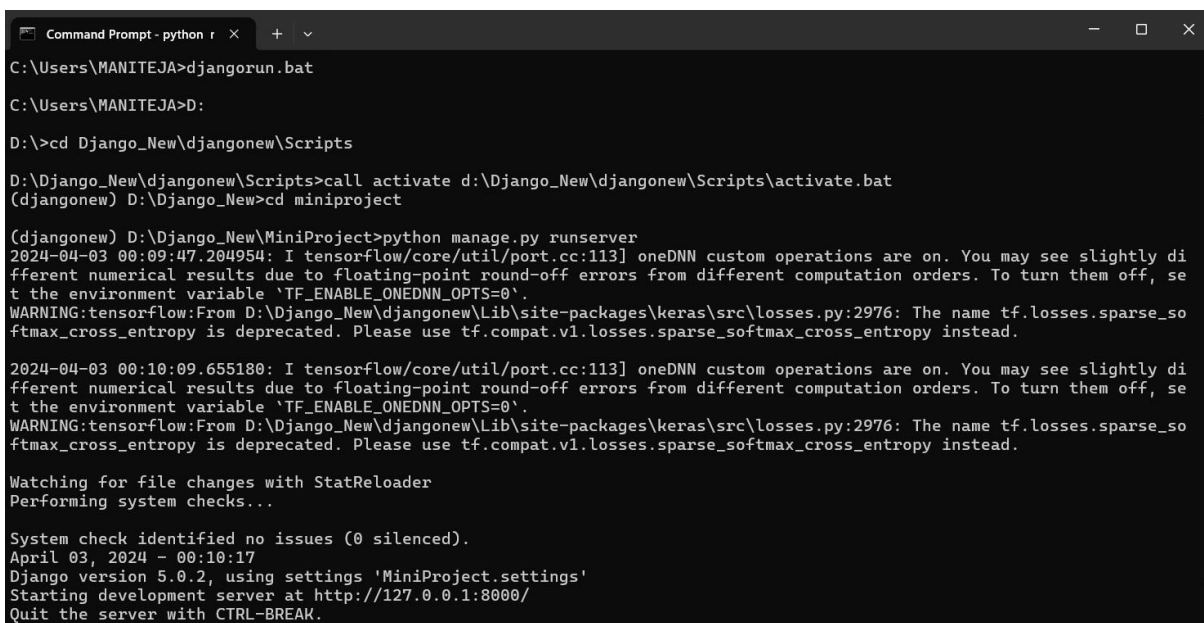
        label=obj.Predict(filename)

        return JsonResponse({'breed': label[0], 'image':img_URL}, status=200)

```

4.5 RESULTS

STEP 1: First Run the Django backend Application using the command `python manage.py runserver`



```

Command Prompt - python r x + v
C:\Users\MANITEJA>djanganorun.bat
C:\Users\MANITEJA>D:
D:\>cd Django_New\djangonew\Scripts
D:\Django_New\djangonew\Scripts>call activate d:\Django_New\djangonew\Scripts\activate.bat
(djangonew) D:\Django_New>cd miniproject
(djangonew) D:\Django_New\MiniProject>python manage.py runserver
2024-04-03 00:09:47.204954: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING:tensorflow:From D:\Django_New\djangonew\Lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

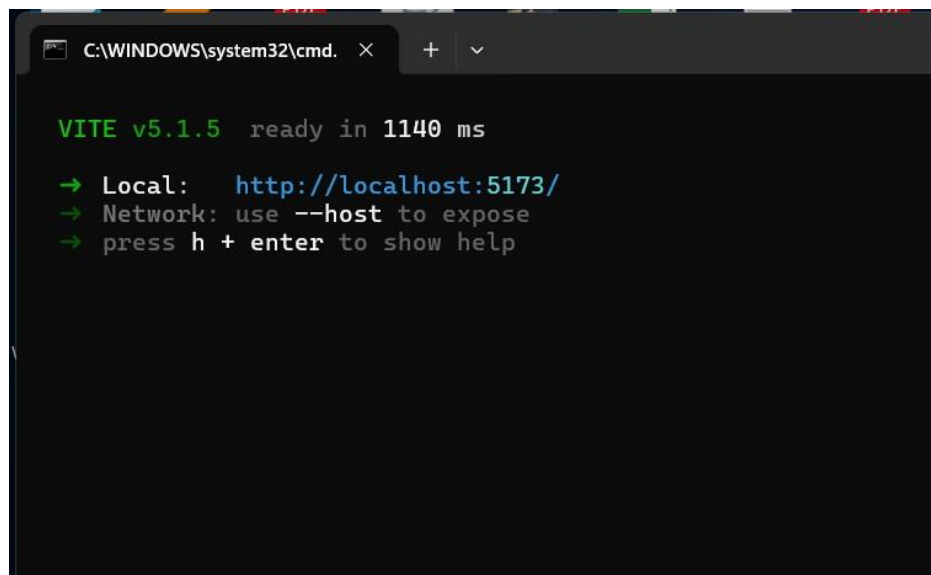
2024-04-03 00:10:09.655180: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING:tensorflow:From D:\Django_New\djangonew\Lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 03, 2024 - 00:10:17
Django version 5.0.2, using settings 'MiniProject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

STEP 2: Run the React Application using the command `npm run dev` in the command prompt



```
C:\WINDOWS\system32\cmd. X + v  
  
VITE v5.1.5 ready in 1140 ms  
→ Local: http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

Fig. 4.1 React Application Execution

STEP 3: Click on the Provided Link which will open the webpage in the Browser

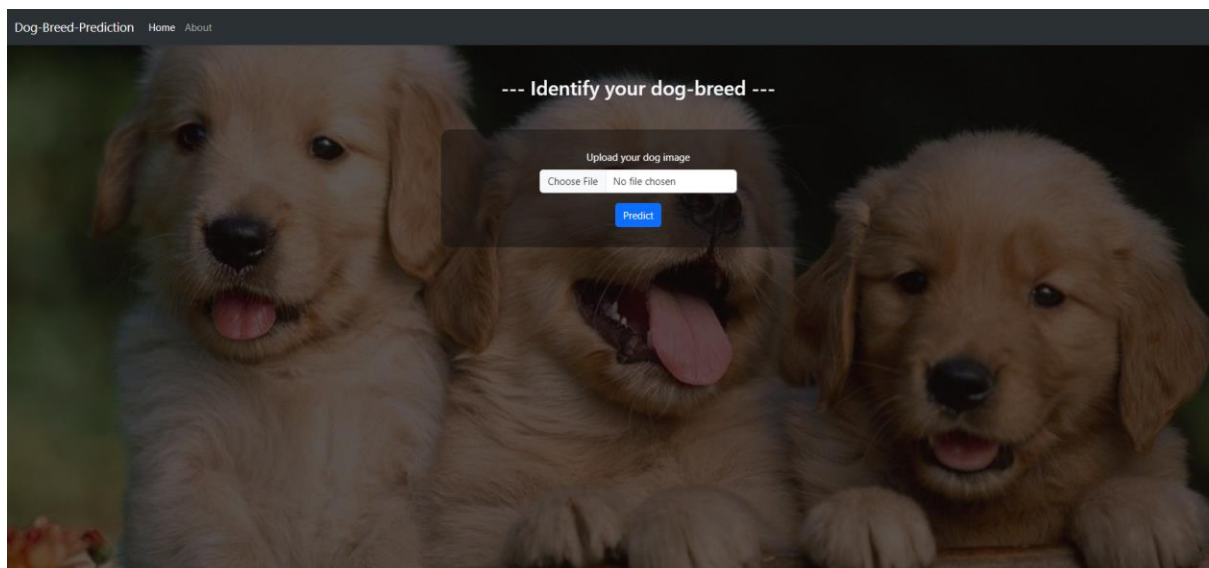


Fig. 4.2 Prediction Model Interface

STEP 4: Click on the choose file and select the dog image to predict

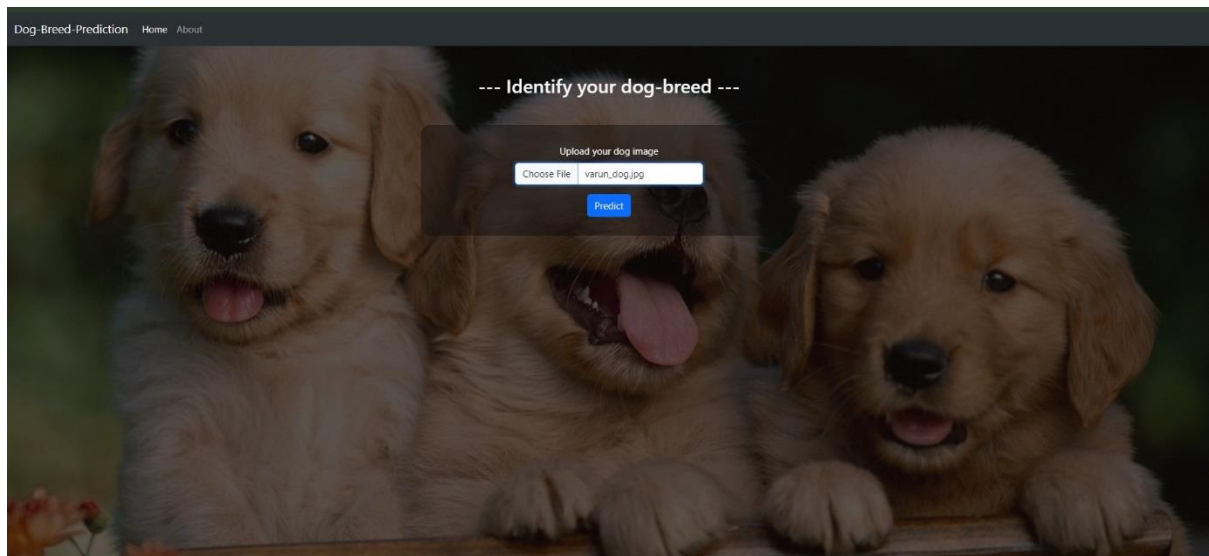


Fig. 4.3 Choosing the Dog Image

STEP 5: Click the Predict Button which predicts the Dog's breed and display it in the same page.

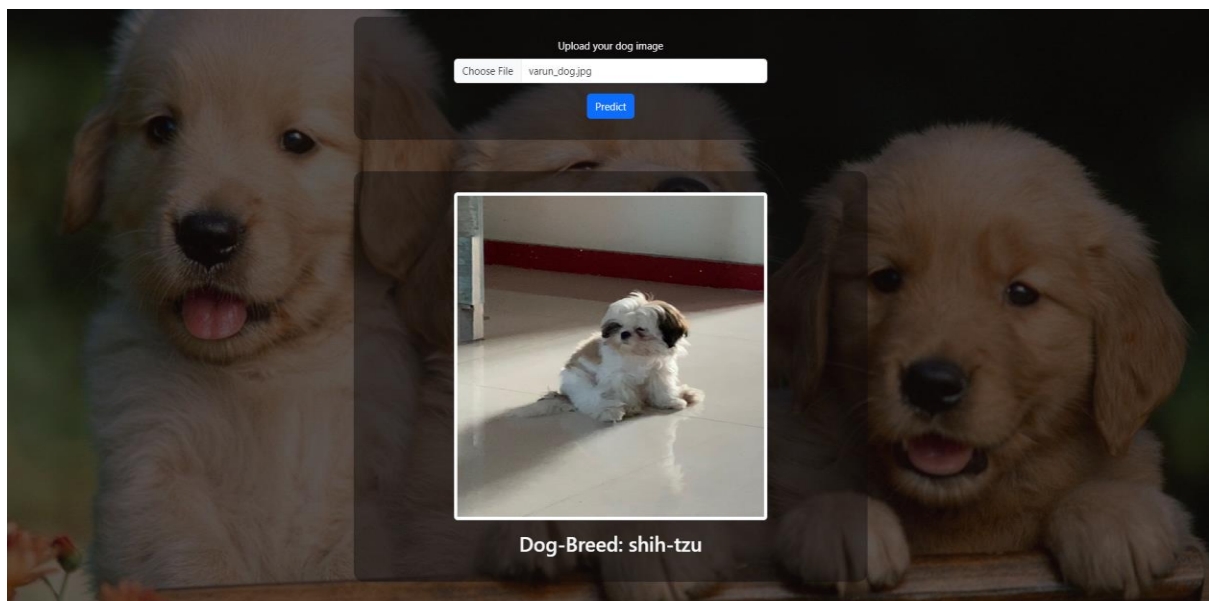


Fig. 4.4 Final Dog's Breed is Predicted

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

Our review made noteworthy progress in fine-grained canine variety characterization, with an amazing exactness pace of more than 92.4%. This precision mirrors our capacity to foresee the right variety with a serious level of certainty, taking into account the huge changeability among the 120 varieties remembered for the dataset. Remarkably, our mixture model, joining Commencement v3 and Xception designs, beat different calculations with the most elevated approval exactness of 97%. These discoveries exhibit the capability of profound learning models in precisely distinguishing canine varieties.

While our mixture model succeeded, we perceive the significance of additional investigation into different AI and profound learning models for canine variety expectation. Future exploration ought to examine elective structures and methods to further develop precision and heartiness. In any case, imperatives, for example, time and computer processor constraints frustrated broad emphases of our procedure utilizing brain organizations. Getting better GPU and gear is urgent for preparing total models and accomplishing higher precision.

Further examination concerning brain networks for central issue location is suggested, zeroing in on various structures and clump iterators to recognize more successful techniques. It's fundamental to recognize the tedious idea of preparing and repeating brain network models, accentuating the requirement for persistence and cautious thought in ongoing undertakings. In spite of their difficulties, brain networks offer strong capacities contrasted with conventional strategies, highlighting their true capacity for propelling canine variety expectation.

5.2 FUTURE SCOPE

In the field of Dog breed prediction, future advancements are promising. Researchers can explore alternative machine learning and deep learning models, seeking novel architectures and techniques to enhance accuracy. Acquiring improved hardware resources, such as powerful GPUs, is pivotal for facilitating extensive model iterations and training. This investment accelerates innovation and enables exploration of sophisticated models, driving progress.

Additionally, research into neural networks for key point detection in dog images offers significant potential. Exploring different network architectures may unveil more effective strategies for improving prediction accuracy. Applying deep learning models in real-world scenarios, like pet identification, provides valuable opportunities for practical validation. Through rigorous testing, researchers can gain insights into real-world performance, paving the way for adoption in diverse applications.

REFERENCES

- [1] Tu, K. Lai, and S. Yanushkevich, “Transfer learning on convolutional neural networks for dog identification,” in Proc. IEEE 9th Int. Conf. Softw. Eng. Service Sci. (ICSESS), Nov. 2018, pp. 357–360, doi: 10.1109/ICSESS.2018.8663718.
- [2] Z.Wang, D. Zhao, and K. Hong, Dog Breed Classification. San Diego, CA, USA: ECE Project Univ. California, 2020.
- [3] A. Varshney, A. Katiyar, A. K. Singh, and S. S. Chauhan, “Dog breed classification using deep learning,” in Proc. Int. Conf. Intell. Technol.(CONIT), Jun. 2021, pp. 1–5, doi: 10.1109/conit51480.2021.9498338
- [4] O.Sinnott, F. Wu, and W. Chen, “A mobile application for dog breed detection and recognition based on deep learning,” in Proc. IEEE/ACM 5th Int. Conf. Big Data Comput. Appl. Technol. (BDCAT), Dec. 2018, pp. 87–96, doi: 10.1109/BDCAT.2018.00019.
- [5] C. Wang, J. Wang, Q. Du, and X. Yang, “Dog breed classification based on deep learning,” in Proc. 13th Int. Symp. Comput. Intell. Design (ISCID), Dec. 2020, pp. 209–212, doi: 10.1109/ISCID51228.2020.00053.